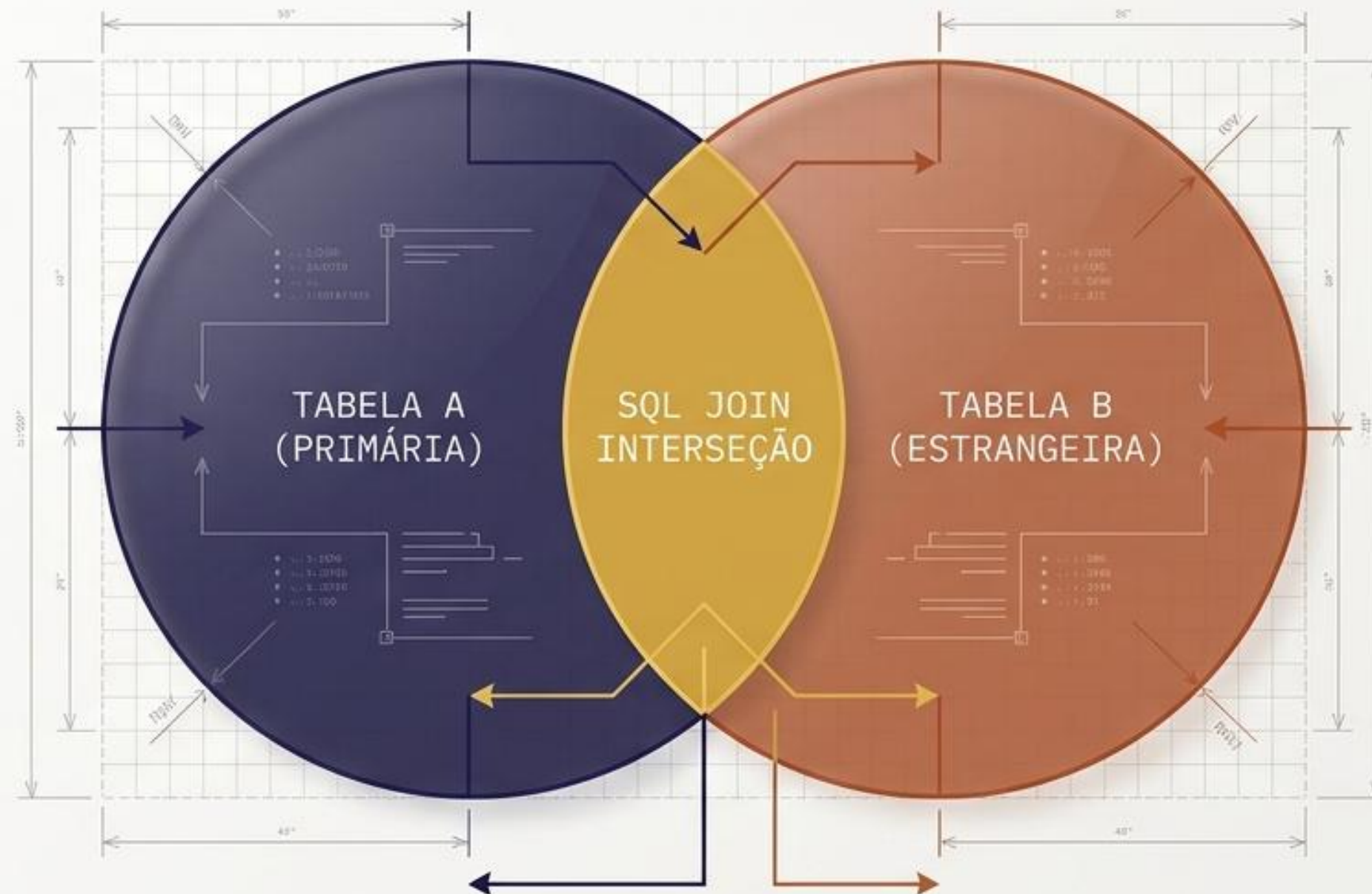


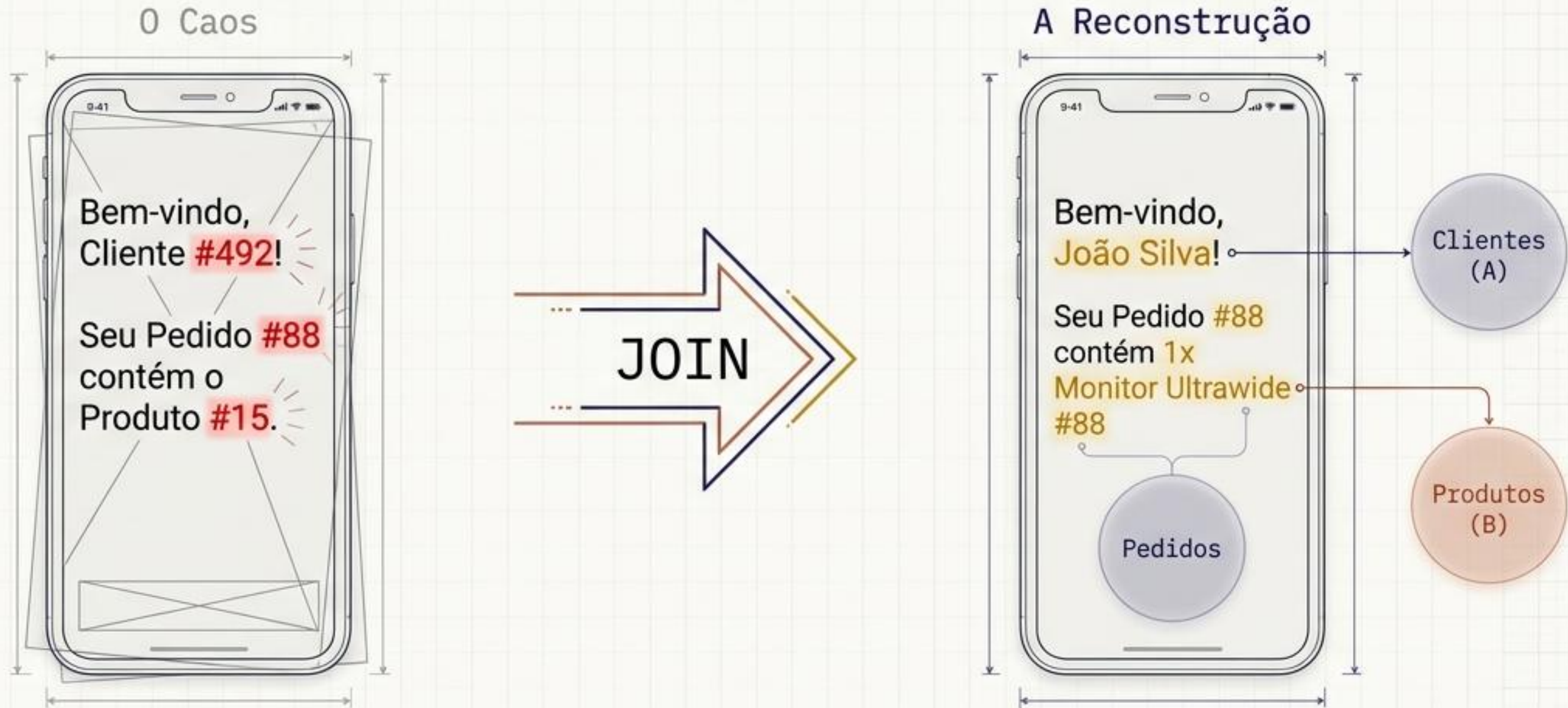
# A Matriz de Cruzamento: Dominando a Lógica dos JOINS

Semana 10 | Ponto Alto da SA2: Reconstruindo a Inteligência dos Dados



# Juntando os Pedacos do Caos

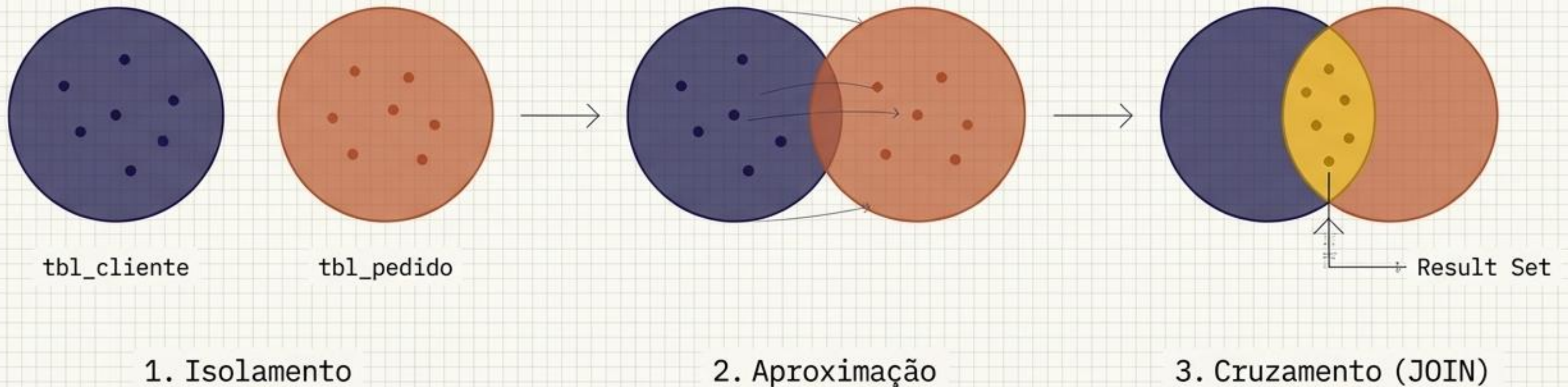
Nas últimas semanas, aplicamos a Normalização (3FN) para proteger o sistema, fragmentando a informação em dezenas de tabelas. Mas o front-end exige semântica: o usuário não lê IDs numéricos. Ele exige nomes e descrições reais.



O banco de dados fala a linguagem das Chaves. O usuário fala a linguagem dos Negócios. JOIN é o tradutor simultâneo.

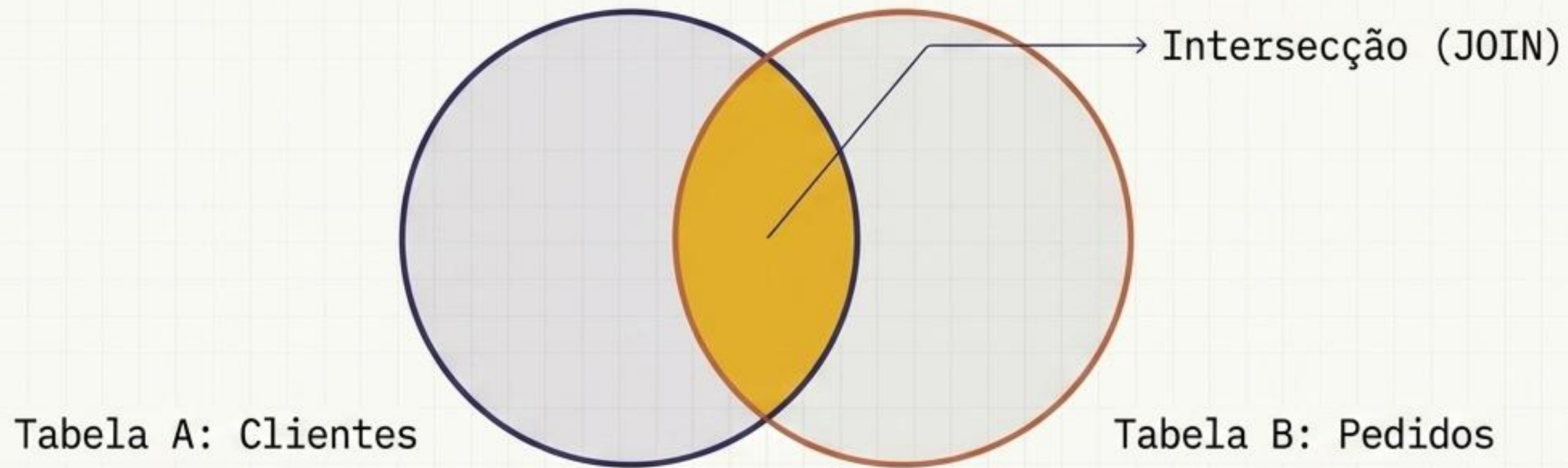
# A Teoria dos Conjuntos Aplicada ao SQL

Para cruzar informações dispersas, o SQL utiliza a lógica matemática dos conjuntos. Visualizamos nossas tabelas como conjuntos de dados que precisam ser sobrepostos e conectados logicamente.



# INNER JOIN: O Casamento Perfeito (Intersecção)

O INNER JOIN é restritivo. Ele retorna estritamente os registros onde há uma correspondência exata entre ambas as tabelas.



```
SELECT c.nome, p.data_compra  
FROM cliente c  
INNER JOIN pedido p
```

Apenas clientes que fizeram pedidos aparecem. Dados sem par perfeito ficam de fora do relatório.

# A Anatomia da Ponte Lógica: Cláusula ON

O SGBD não sabe como as tabelas se relacionam apenas pelo comando JOIN. Precisamos construir uma ponte explícita, ligando a Chave Primária (PK) de uma tabela à Chave Estrangeira (FK) da outra.

**ON** cliente.id\_cliente = pedido.fk\_cliente



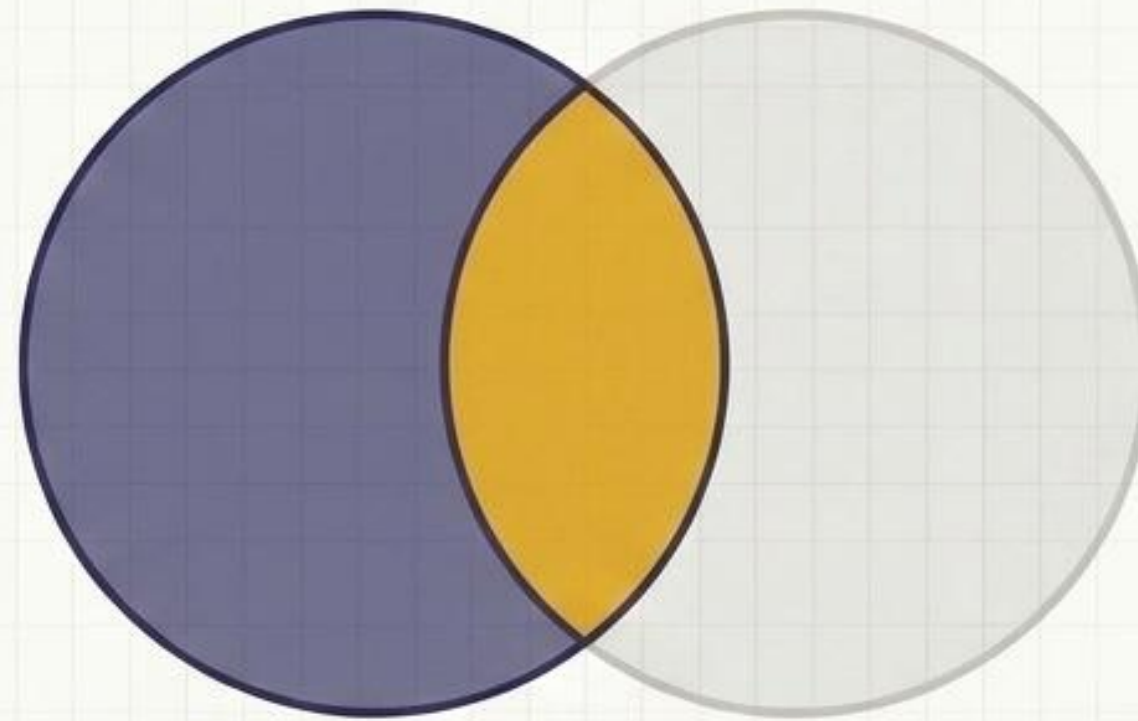
A Fechadura - PK



A Chave - FK

# LEFT JOIN: Preservando a Origem

Diferente da intersecção estrita, o LEFT JOIN força a tabela "da esquerda" a mostrar 100% dos seus registros. Se não houver correspondência na direita, o espaço é preenchido com valores nulos.

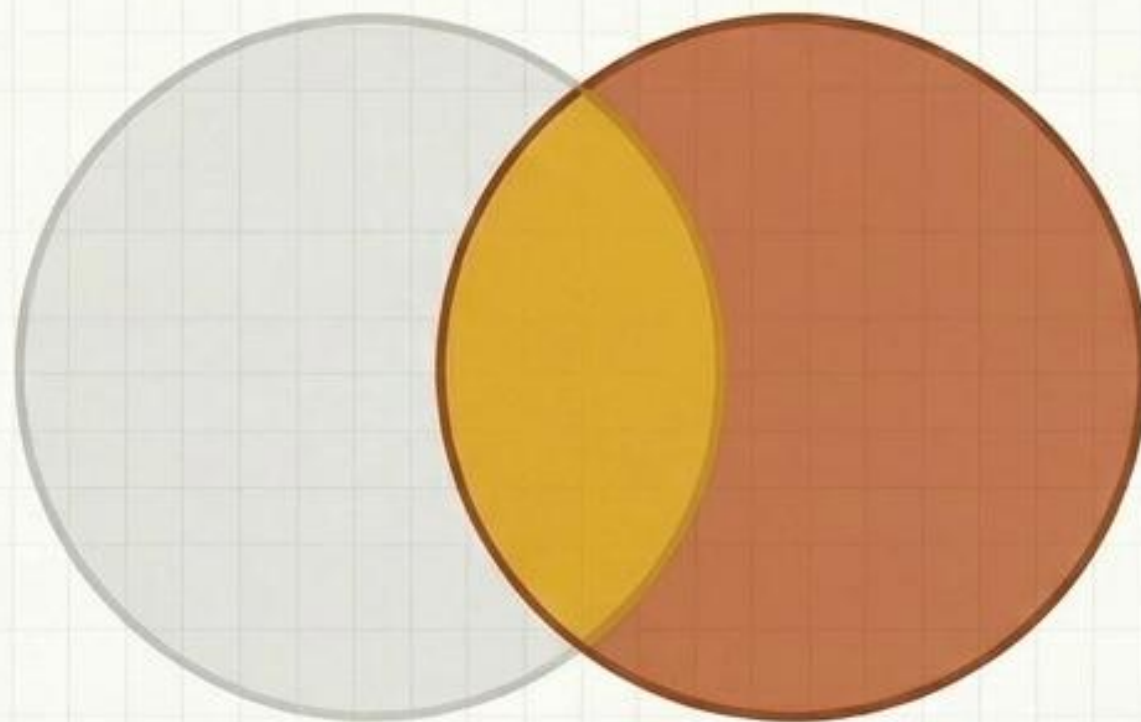


```
FROM cliente c  
LEFT JOIN pedido p
```

Se o João nunca comprou nada, ele ainda aparece na lista final, e a coluna do pedido mostrará a tag **[NULL]**

# RIGHT JOIN: Preservando o Destino

O RIGHT JOIN aplica a exata mesma lógica de preservação, mas invertendo a prioridade para a tabela “da direita” (a segunda declarada no código).

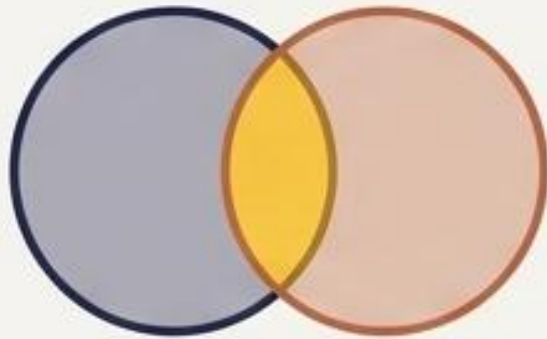


## Dica de Arquitetura:

Na prática profissional, raramente utilizamos o RIGHT JOIN. Por convenção de leitura ocidental (da esquerda para a direita), é estruturalmente mais elegante inverter a ordem das tabelas e utilizar apenas o LEFT JOIN.

# Matriz de Decisão de Conjuntos

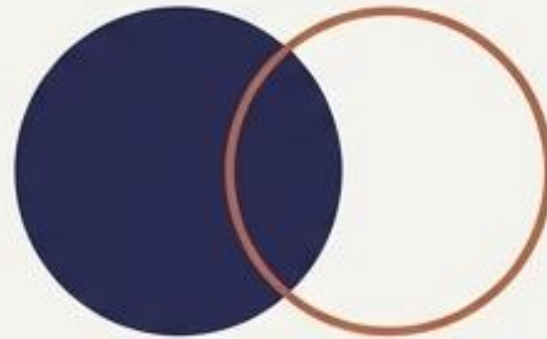
## INTERSECÇÃO



Traz apenas os pares perfeitos.

**INNER JOIN**

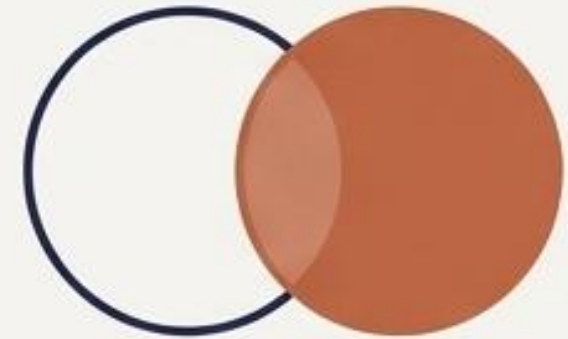
## PRESERVAÇÃO PRIMÁRIA



Traz tudo da Tabela A, aceita vazios da Tabela B.

**LEFT JOIN**

## PRESERVAÇÃO ESTRANGEIRA



Traz tudo da Tabela B, aceita vazios da Tabela A.

**RIGHT JOIN**

# Atividade Guiada: Ligue os Pontos Lógicos

Antes de encostar no código, vamos testar o pensamento analítico.

| Alunos        |
|---------------|
| ID 1 (Ana)    |
| ID 2 (Beto)   |
| ID 3 (Carlos) |

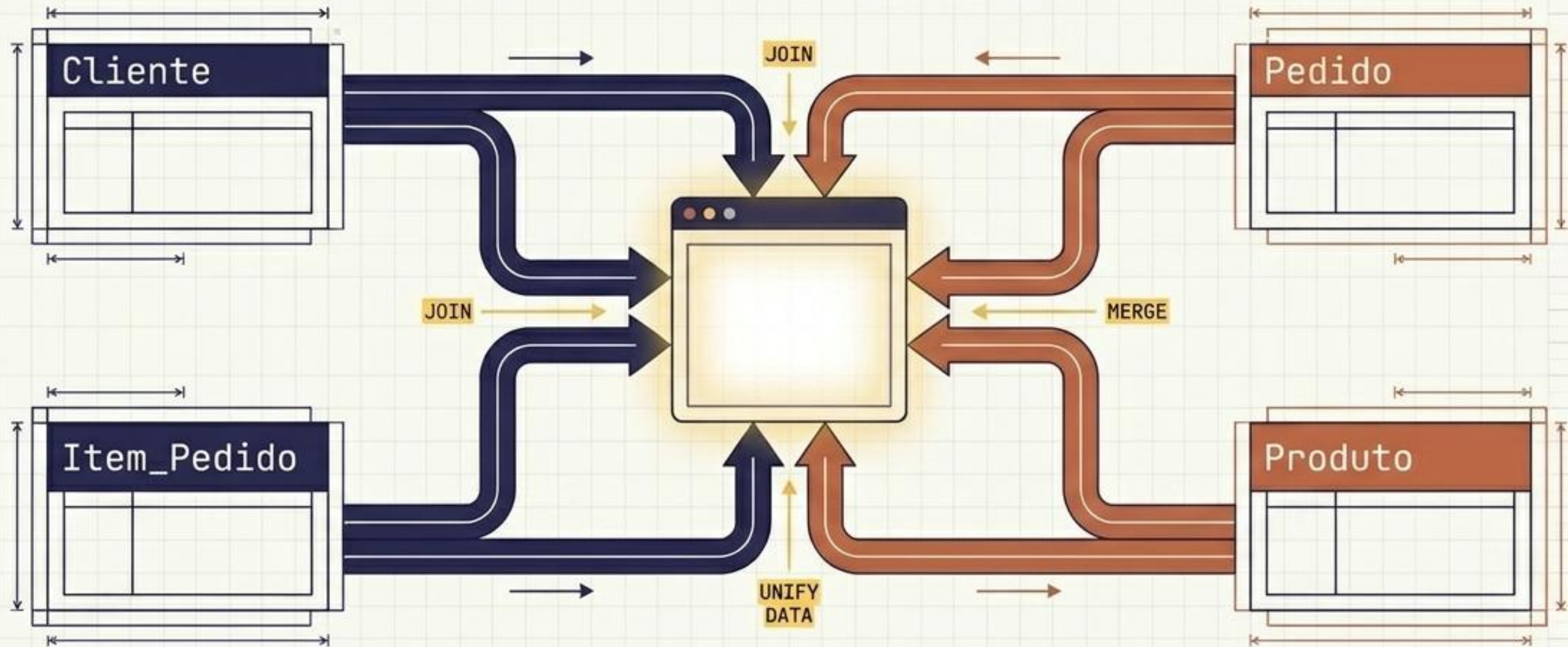
| Cursos                      |
|-----------------------------|
| ID_Aluno 1 (Banco de Dados) |
| ID_Aluno 3 (Java)           |

1. Se rodarmos um **INNER JOIN**: 2 linhas retornam (Ana e Carlos)

2. Se rodarmos um **LEFT JOIN** partindo de **Alunos**: 3 linhas retornam (Ana, Beto com NULL, e Carlos)

# Bloco B: O Grande Cruzamento

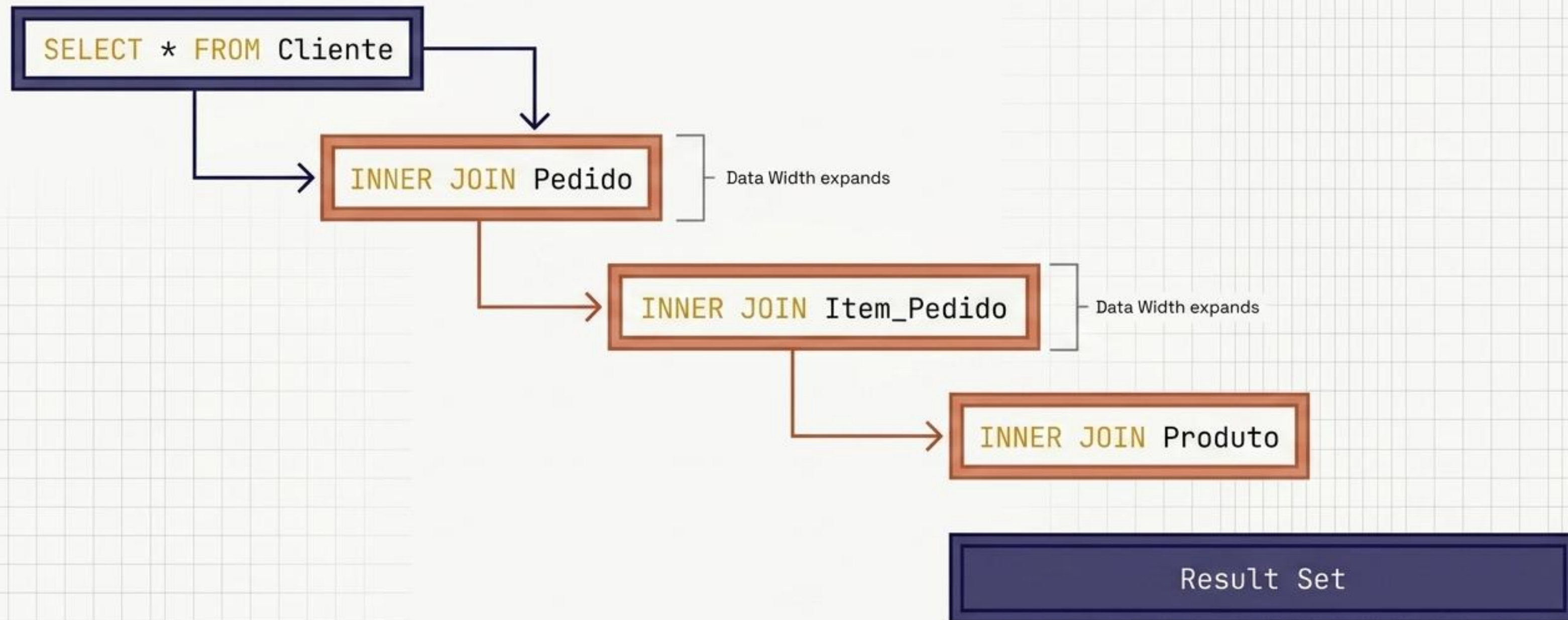
A tela de 'Detalhes do Pedido' está em branco. Os dados vitais estão estilhaçados em quatro tabelas separadas. Sua missão como Arquiteto de Dados é criar a Query Perfeita que reconstrua este ecossistema.



# Pipeline: Encadeando Múltiplos JOINS

O SQL não tem limite de cruzamentos.

Visualize o código como um pipeline em cascata. Cada novo JOIN engorda o conjunto de resultados horizontalmente.



# A Obrigatoriedade dos Apelidos (Table Aliases)

Ao cruzar múltiplas tabelas, o código pode se tornar um pesadelo ilegível. A técnica do Table Alias renomeia as tabelas apenas durante a execução, garantindo um código enxuto e profissional.

## → O Pesadelo

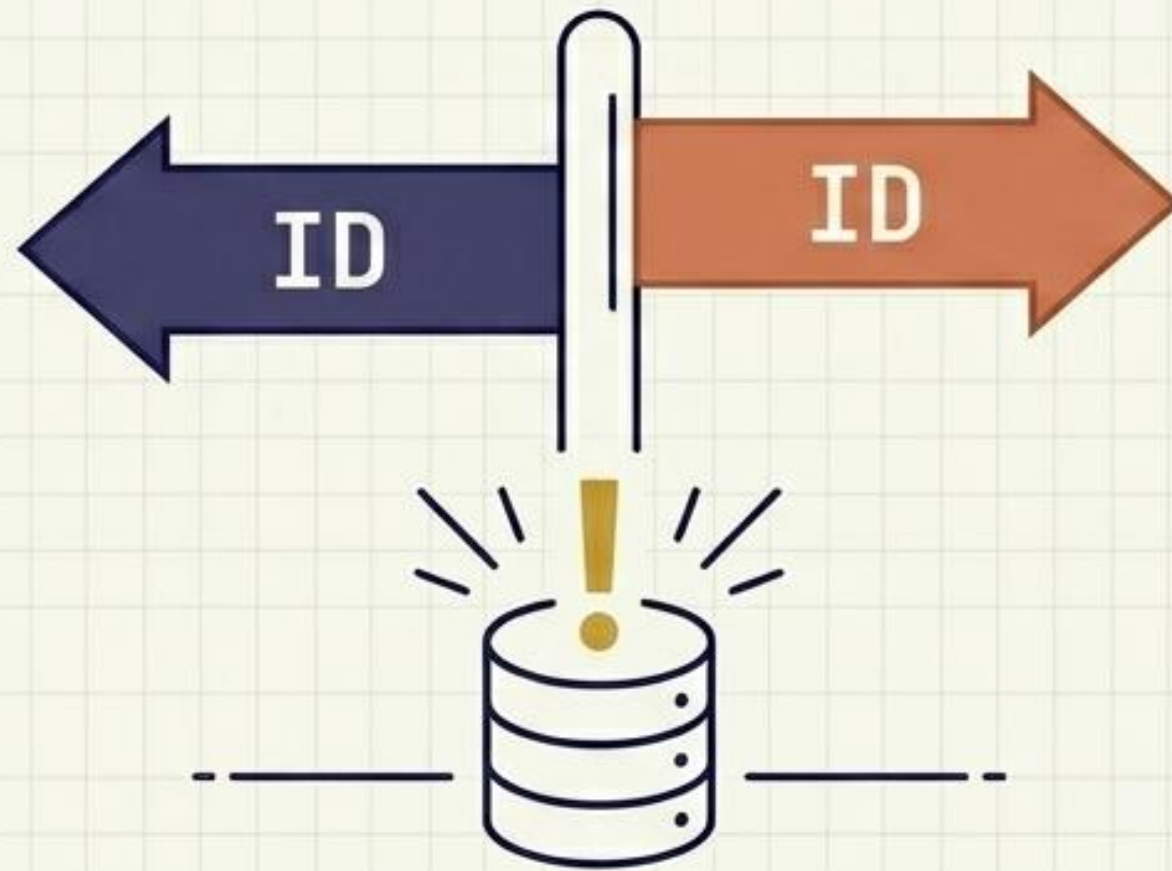
```
SELECT tbl_cliente_ecommerce.nome,  
tbl_produto_estoque.descricao  
FROM tbl_cliente_ecommerce  
INNER JOIN tbl_pedido_venda  
ON tbl_cliente_ecommerce.id =  
tbl_pedido_venda.fk_cliente
```

## → A Mestria

```
SELECT c.nome, pd.descricao  
FROM cliente c  
INNER JOIN pedido pd  
ON c.id = pd.fk_cliente
```

# O Ponto de Bifurcação: Erro Ambiguous Column

Quando cruzamos tabelas, muitas vezes ambas possuem colunas com o mesmo nome exato (como 'id'). O banco de dados não tem bola de cristal; você precisa usar o Alias como prefixo.

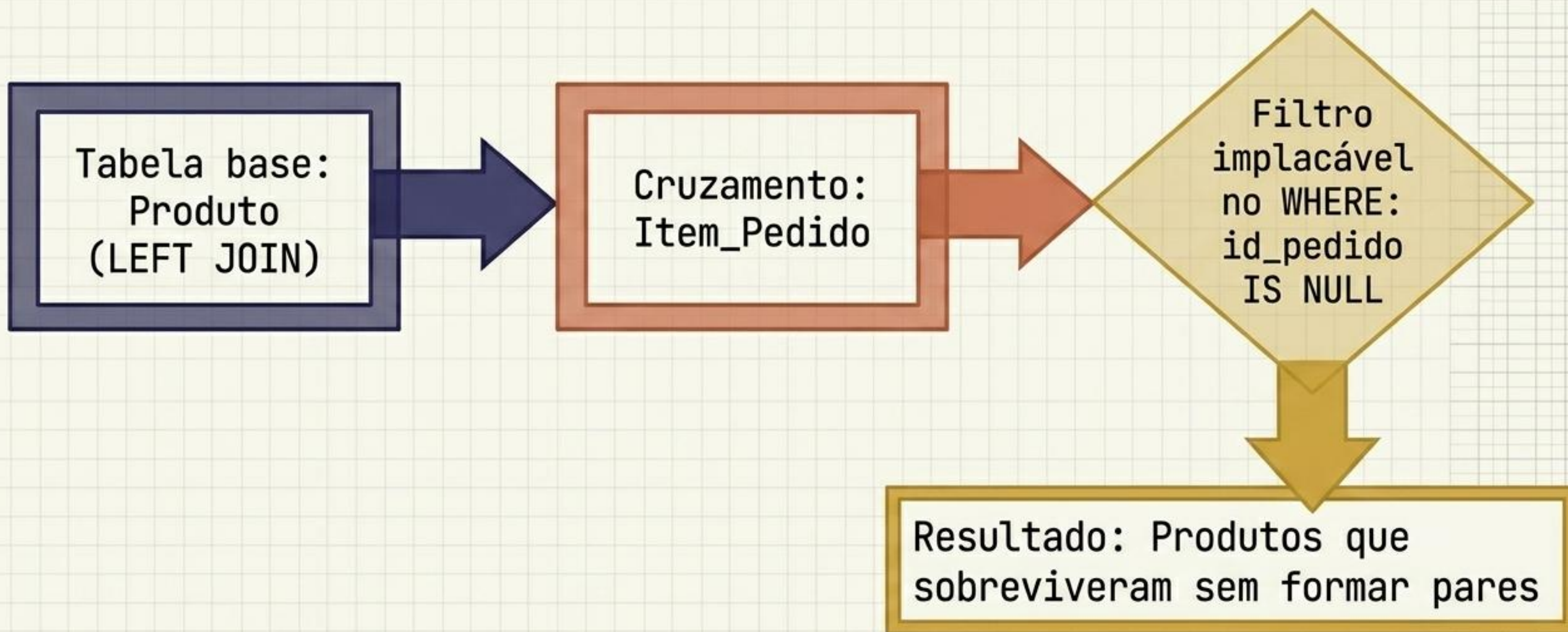


**ERRO FATAL:**  
Ambiguous column name 'id'

Correção obrigatória:  
Use **c.id** ou **p.id** na sua projeção.

# Tarefa Extra 1 (Nível AUT): O Caçador de Órfãos

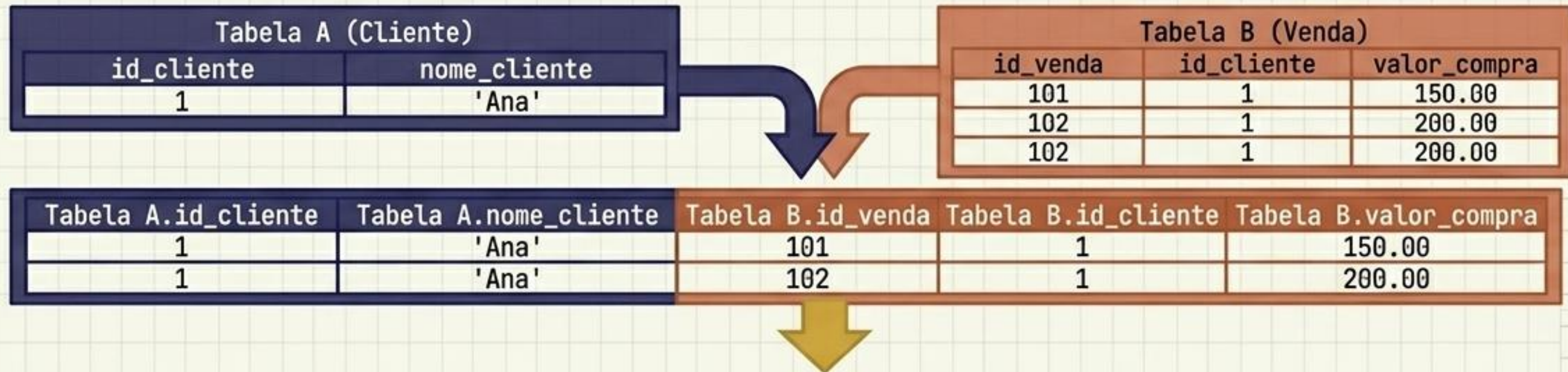
Missão Analítica: O setor de compras precisa saber 'Quais produtos nunca foram vendidos?'. Um INNER JOIN ignoraria esses itens. Precisamos aplicar investigação forense usando a preservação.



# Tarefa Extra 2 (Nível AUT): O Mestre da Agregação

Inteligência de Negócios exige resumos complexos. Desafio: “Mostre o nome do Cliente e a soma total de R\$ que ele já gastou”. A ordem de execução interna do banco de dados importa.

Passo 1: INNER JOIN (Reúne o nome na Tabela A e os valores na Tabela B)



Passo 2: GROUP BY + SUM() (Esmaga as múltiplas compras em um totalizador)



## Tarefa Extra 3 (Nível AUT): Alias Mastery

É hora de refatorar e polir. Como Arquiteto de Dados Sênior, você deve reescrever a query de 4 JOINS aplicando os mais altos padrões corporativos de organização de código.



Substituir todos os nomes longos de tabelas por Aliases concisos de 1 ou 2 letras (ex: `<cliente c</code>`).



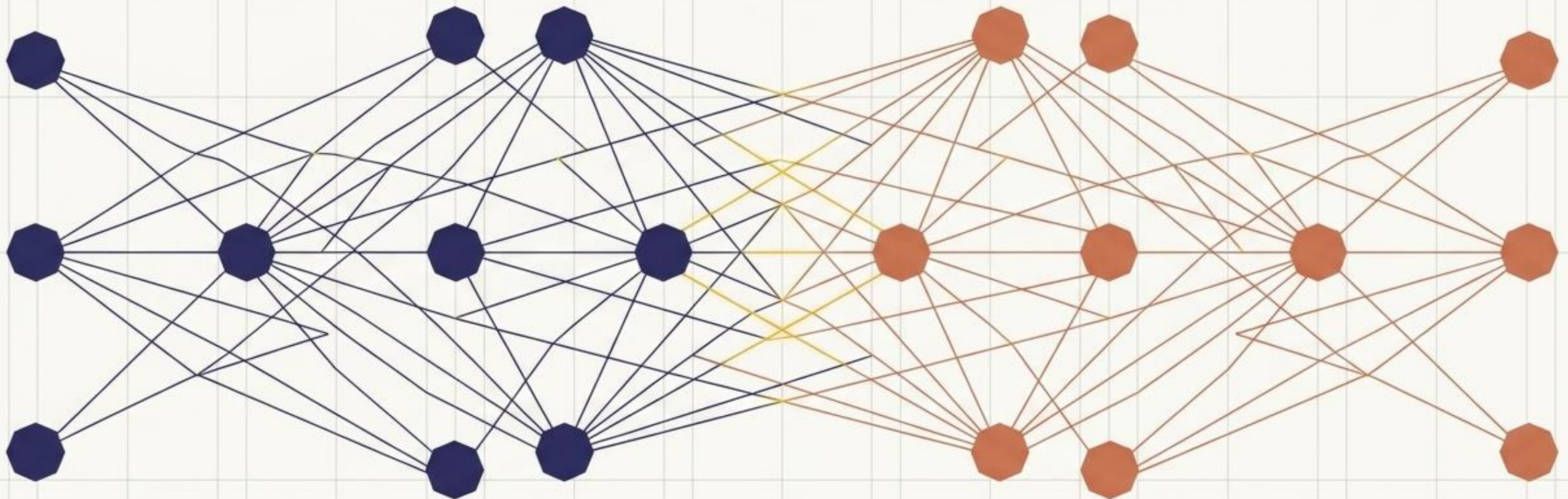
Garantir que absolutamente TODAS as colunas no SELECT contenham seus prefixos de origem (ex: `<c.nome</code>`, `<code>ip.quantidade</code>`).



Formatar o código respeitando a indentação vertical das cláusulas para facilitar a leitura da equipe (CS 1).

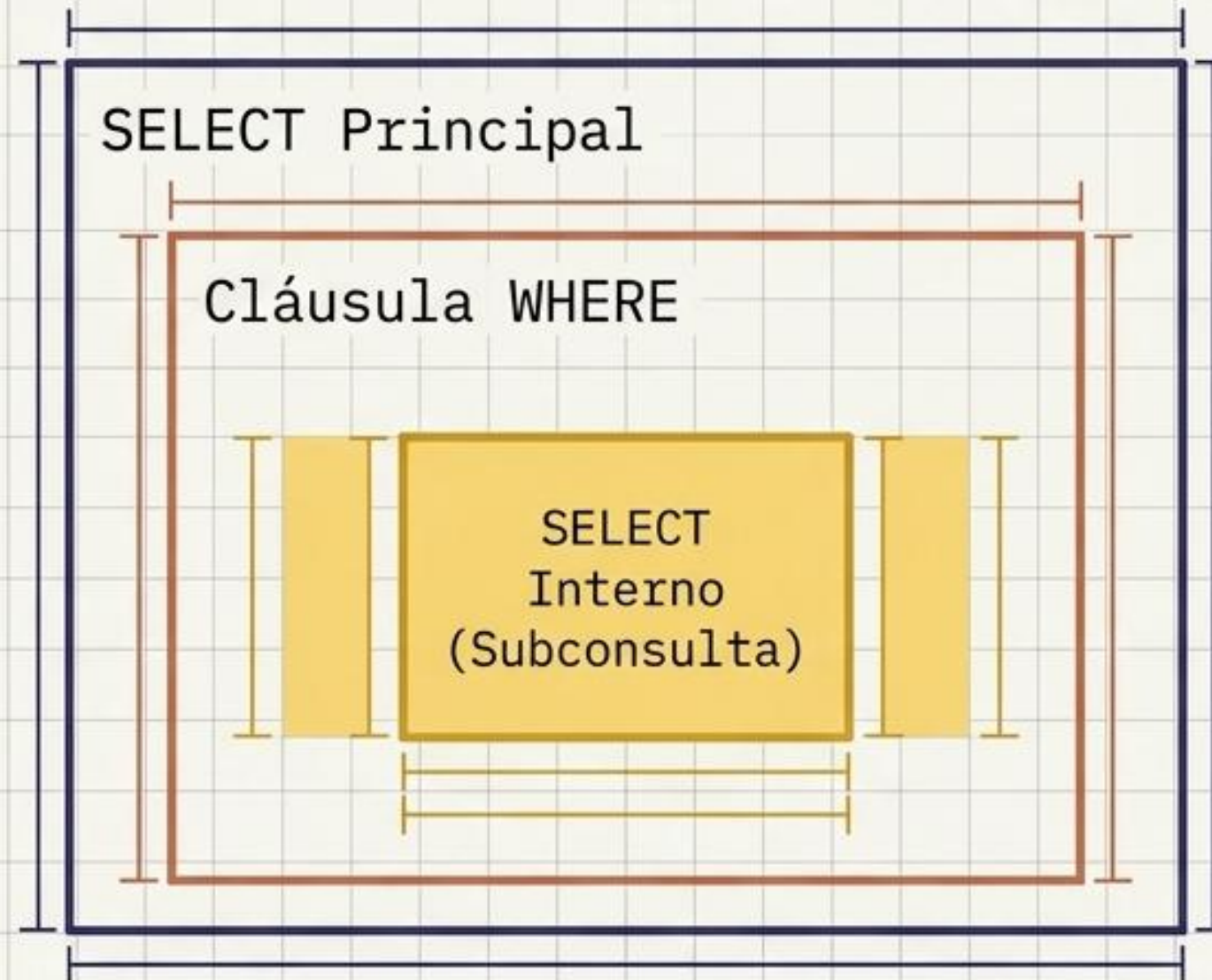
# Status do Sistema: Lógica Reconstruída

Parabéns. O caos da normalização agora tem ordem. O domínio dos cruzamentos (JOINS) separa vocês dos amadores, sendo o requisito número um para qualquer vaga de backend e arquitetura de dados.



# Próximos Passos: Inception de Dados

Vocês conseguiram recompor os dados horizontalmente. Mas às vezes, a pergunta de negócio é tão complexa que não pode ser resolvida com cruzamentos diretos. Precisaremos embutir uma pesquisa inteira DENTRO de outra pesquisa.



**Semana 11 | Subconsultas. Preparem-se.**